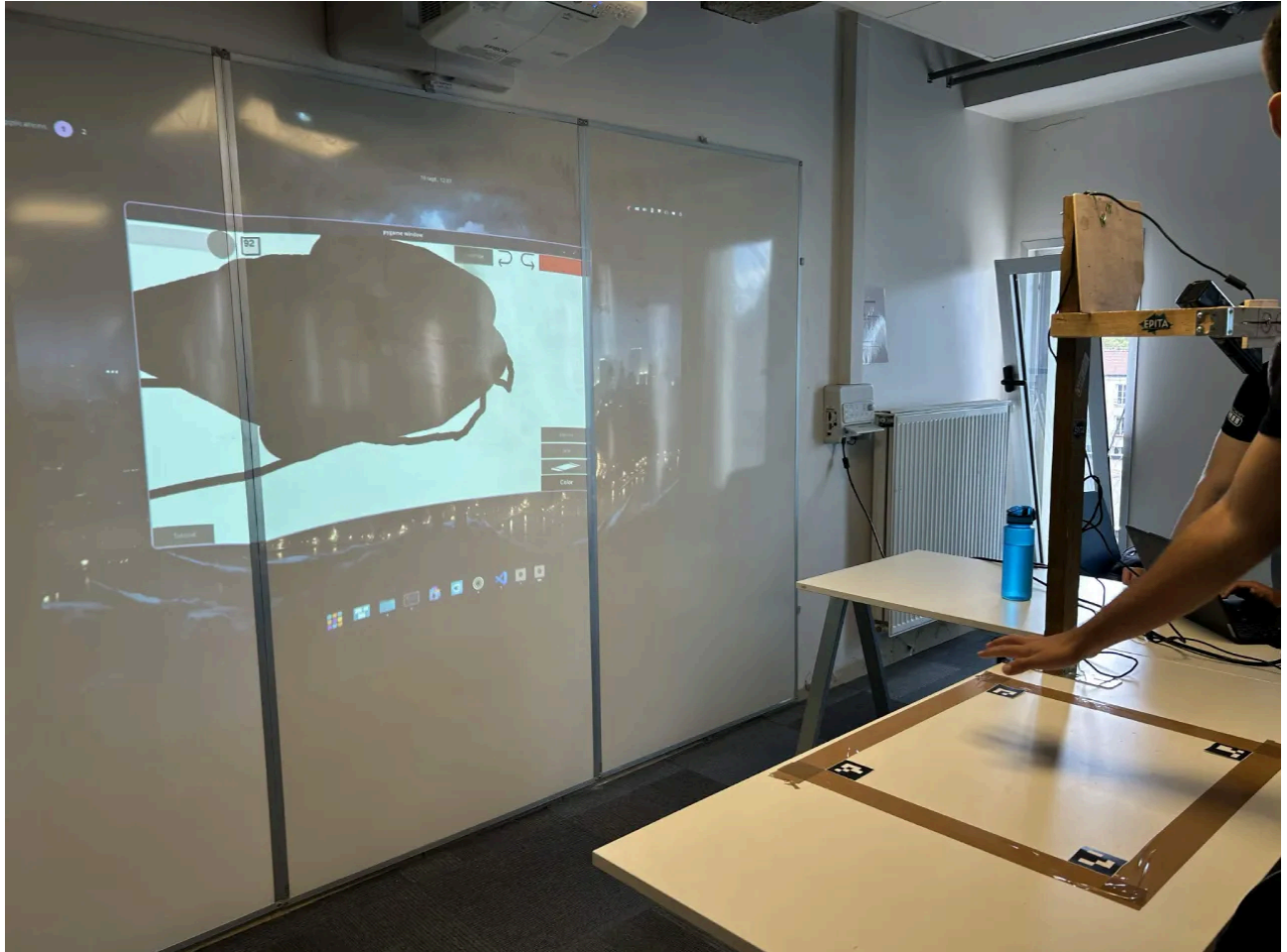


Documentation Technique

Idée Générale du Produit :

Notre projet est une application de dessin interactive inspirée de Paint. L'utilisateur dessine sur sur table et son dessin est affiché une surface projetée face à lui, un tableau par exemple, en utilisant son doigt détecté via une Kinect. La Kinect fournit les informations nécessaires à la détection de l'utilisateur, de ses mouvements et de la hauteur.



Lancement avec Docker :

...

```
./start.sh
```

...

L'utilisation la plus facile est de lancer l'application avec le docker. Si vous souhaitez lancer séparément les outils, orientez-vous vers le lancement avec Nix.

Lancement avec Nix:

```
...  
cd canvas_loop/  
uv run main.py  
...
```

Kinect utilise libfreenect qui a une installation complexe, facilité si on passe par nix pour installer les paquets.

```
...  
cd kinect/  
nix develop  
cmake -S . -B build  
cd build/  
make  
./test-cv  
...
```

Installation Physique:

L'installation nécessite une table, un vidéo projecteur et une kinect accrochée à un mât.

On délimite un rectangle avec les 4 marquages spécifiques (aruco marker) qui donnent la position des coins pour aider la kinect à reconnaître la zone.

Une calibration est nécessaire, et doit se faire sans main sur la zone de jeu. Les marquages ont leur ordre noté sur l'écran.

Pour nos tests nous avons utilisés un rectangle de 46 cm x 56 cm

On fixe la kinect avec le mât à côté du rectangle en orientant la kinect vers la zone ciblée

Il suffit ensuite de connecter le vidéo projecteur et la kinect à l'ordinateur, puis de projeter l'output sur un tableau / écran.

Modules Logiciels :

L'architecture repose sur deux modules :

- Le module Kinect, développé en C++,
- L'application graphique, développée en Python.

Le module Kinect est lié au dossier 'kinect' et toute l'application graphique, la calibration et le traitement du flux vidéo est liée au dossier 'canvas_loop'.

Choix Techniques :

L'implémentation de l'application fut réalisée en Python. Pour le jeu lui-même, nous utilisons PyGame pour le rendu graphique. Nous avons fait ce choix pour une implémentation plus

rapide. En effet, la bibliothèque Pygame permet d'implémenter un jeu rapidement et facilement (création d'une fenêtre, affichage des images, dessin des lignes, gestion des couleurs, etc).

Pour l'utilisation et la communication avec la Kinect, nous utilisons la bibliothèque LibFreeNect. Nous utilisons cette bibliothèque car elle permet de faire un lien facile avec la Kinect. De plus, la code base donnée utilisait déjà cette bibliothèque, c'est pourquoi nous avons gardé son utilisation. OpenCV est aussi utilisé dans le module Kinect.

Compilation:

En C++, nous compilons simplement avec l'outil 'CMake'.

En Python, nous utilisons le gestionnaire de paquets 'uv'.

Les problèmes rencontrés:

Il y a eu des principalement des problèmes de reconnaissance des doigts. Afin d'avoir la meilleure expérience possible, nous conseillons de resserrer les doigts entre eux et d'éviter les mouvements rapides.

Les pivots réalisés:

Un des pivots majeurs fut le changement de projection du dessin. Notre idée initiale était de projeter le dessin là où l'utilisateur dessinait (directement sur la table). Nous avons fait le choix de séparer l'espace de dessin de l'image projetée. L'image projetée est maintenant en face de l'utilisateur (sur un tableau) alors que l'espace de dessin reste sur la table.

Ajustements à faire :

Des ajustements sur la possibilité de faire du travail collectif simultané (ex : fresque collective) sont à prévoir. De plus, des ajustements sur la sensibilité du calibrage est à prévoir, afin de perfectionner l'application.